

Random Forests (Rastgele Ormanlar) Algoritması

Dr.Öğr.Üyesi Günay TEMÜR

Random Forests (Rastgele Ormanlar) Algoritması Nedir?

- Karmaşık bir problemi çözmek ve modelin performansını iyileştirmek için birden fazla sınıflandırıcıyı birleştirme süreci olan topluluk öğrenme kavramına dayanır.

Karar Ağaçlarından Ensemble Modellerine Geçiş

- Decision Tree'nin bazı problemleri vardır:
 - Overfitting yapabilir
 - Veri değişimine karşı hassas olabilir
 - Tek ağaç her zaman güçlü olmayabilir
- Bu problemi çözmek için:
- 👉 **Ensemble yöntemleri geliştirilmiştir**

Ensemble Learning Nedir?

- Ensemble learning:
 - Birden fazla modelin birlikte çalışarak daha güçlü bir tahmin üretmesidir.
- Mantık:
 - Tek model → daha fazla hata

 - Birçok model → daha kararlı sonuç
- Örnek:
 - Model 1 → Evet
 - Model 2 → Hayır
 - Model 3 → Evet

 - Sonuç → Evet
- Bu yöntem:
- 👉 **Majority Voting (çoğunluk oylaması)** denir.

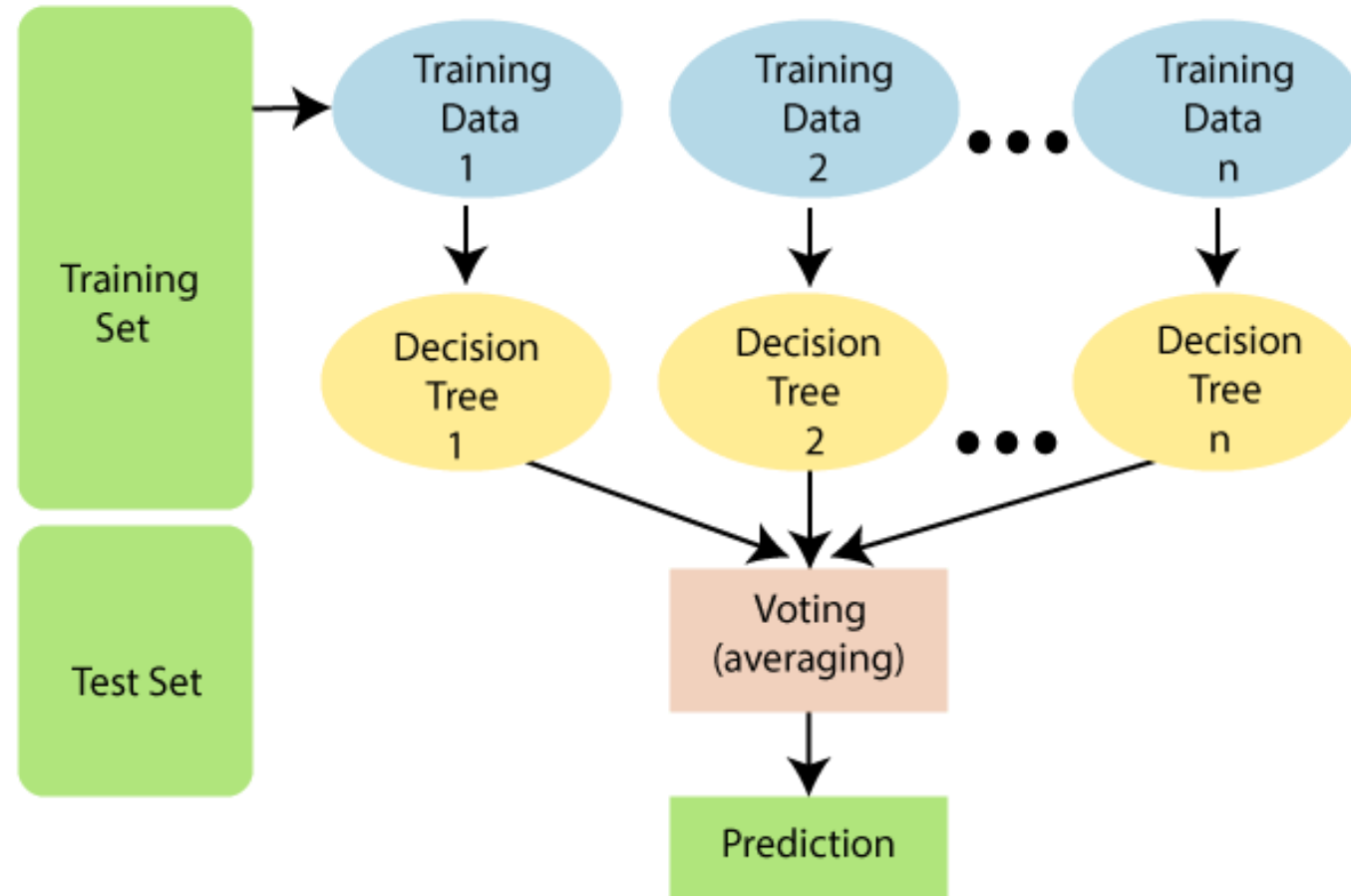
Random Forest Nedir?

- Random Forest:
 - Çok sayıda Decision Tree'nin birlikte çalıştığı bir modeldir.
- Yapısı:
 - Veri
 - ↓
 - Birçok Decision Tree
 - ↓
 - Tahminler
 - ↓
 - Ortalama / Oylama
 - ↓
 - Final sonuç

• Random Forest Avantajları

- ✓ Overfitting azalır
- ✓ Daha güçlü model
- ✓ Daha stabil sonuçlar
- ✓ Feature importance hesaplanabilir

Random Forests



Random Forests

- Algoritma, 5 adımdan oluşur.
- Adım 1- Eğitim setinden rastgele K veri noktaları seçilir.
- Adım 2- Seçilen veri noktalarıyla ilişkili karar ağaçları oluşturulur.
- Adım 3- Oluşturulmak istenilen karar ağaçları kadar N sayısı seçilir.
- Adım 4- Adım 1 ve 2 tekrarlanır.
- Adım 5- Yeni veri noktaları için, her karar ağacının tahminleri bulunur ve yeni veri noktaları çoğunluk oyu kazanan kategoriye atanır.

Random Forests

- Bankacılık sektörü, kredi riskinin tespiti için çoğunlukla bu algoritma kullanılmaktadır. Ya da tıp sektöründe, bu algoritma yardımıyla hastalık eğilimleri ve hastalığın riskleri belirlenebilir.

Decision Tree ve Random Forest Aynı Amaç İçin mi Çalışıyor?

- **Evet**, ikisi de **aynı problemi çözmeye çalışıyor**:

👉 *Verilen özelliklerden (yaş, maaş, mesai durumu vb.) yola çıkarak, çalışanın işten ayrılıp ayrılmayacağını (Attrition) tahmin etmek.*

Özellik	Decision Tree (Karar Ağacı)	Random Forest
Yapı	Tek bir ağaç	Birçok ağacın birleşimi (bir orman)
Eğitim	Hızlı ve basit	Biraz daha uzun (çok ağaç var)
Overfitting (Ezberleme)	Kolayca ezberleyebilir	Ezberlemeye karşı daha dayanıklı
Doğruluk	Genellikle daha düşük	Genellikle daha yüksek
Yorumlama	Çok kolay (ağaç şeması)	Bireysel ağaçlar anlaşılır, ama ormanın tamamı karmaşık
Kullanım Durumu	Küçük veri setlerinde açıklayıcı modeller için	Daha doğru, büyük veri setlerinde veya karmaşık problemlerde

Özet

Karşılaştırma	Decision Tree	Random Forest
Ağaç sayısı	1	100+
Veri kullanımı	Tüm veri	Her ağaç farklı veri dilimleriyle
Özellik kullanımı	Tüm özellikler	Her ağaç farklı özellik setleriyle
Dayanıklılık	Daha az	Daha çok (ensemble learning)
Doğruluk	Genellikle daha düşük	Genellikle daha yüksek

Uygulama Adımları

Adım	Açıklama
1	Gerekli kütüphaneler (pandas, numpy, matplotlib, seaborn, sklearn) yükleniyor.
2	CSV dosyasından veri okunuyor (pd.read_csv).
3	Verinin ilk görünümü (df.head()) ve işten ayrılma (Attrition) oranı görselleştiriliyor.
4	İşi kolaylaştırmak için gereksiz sütunlar (EmployeeCount, EmployeeNumber, vb.) kaldırılıyor.
5	Kategorik değişkenler (örneğin Departman Adı gibi) listeleniyor.
6	Attrition hedef değişkeni, 0 ve 1 gibi sayısal koda dönüştürülüyor.
7	Diğer kategorik değişkenler de sayısal hale (LabelEncoder) getiriliyor.
8	Veri eğitim (%70) ve test (%30) olarak bölünüyor (train_test_split).
9	Model performansını ölçmek için bir değerlendirme fonksiyonu (print_score) tanımlanıyor.
10	Karar Ağacı (DecisionTreeClassifier) modeli kuruluyor ve eğitiliyor.
11	Karar Ağacı modeli için ağaç görselleştirmesi yapılıyor.
12	Random Forest (RandomForestClassifier) modeli kuruluyor ve eğitiliyor.
13	Random Forest içindeki tek bir ağaç seçilip görselleştiriliyor.
14	Random Forest'ta özellik önem dereceleri (hangi değişkenler tahminde daha etkili?) grafikte gösteriliyor.

Sonuç

- **Karar Ağacı** basit bir modelle işe kimlerin devam edip etmeyeceğini öğrenmeye çalışıyor.
- **Random Forest** ise birden çok ağaç birleştirilerek daha sağlam tahminler yapıyor.
- Özellik önem dereceleri çıkarılarak:
 - Mesela “OverTime” (fazla mesai yapanlar mı daha çok ayrılıyor?) gibi bilgiler öğreniliyor.

K-Nearest Neighbors

(KNN) K-en yakın komşu algoritması

K-Nearest Neighbors

- K-en yakın komşu (k-nearest neighbors, KNN) algoritması, gözlemlerin birbirlerine olan benzerlikleri üzerinden tahminlerin yapıldığı, denetimli öğrenmede sınıflandırma ve regresyon için kullanılan algoritmalardan biridir. En basit makine öğrenmesi algoritması olarak kabul edilir.



K-Nearest Neighbors

- Sınıfı bilinmeyen verilerin, eğitim setindeki diğer veriler ile karşılaştırılıp bir uzaklık ölçümü gerçekleştirilmesi sonucu hesaplanan uzaklığa göre henüz bir sınıfa atanamamış verinin, en ideal (optimal) sınıfa atanarak sınıflandırılması olarak ifade edilmektedir.

K-Nearest Neighbors

- KNN'nin temel mantığı şudur:
 - Yeni gelen bir veri, kendisine en yakın olan komşuların sınıfına göre sınıflandırılır.
- Yani algoritma şunu sorar:
 - “Bu yeni gözlem, daha önce gördüğüm hangi örneklerle benziyor?”

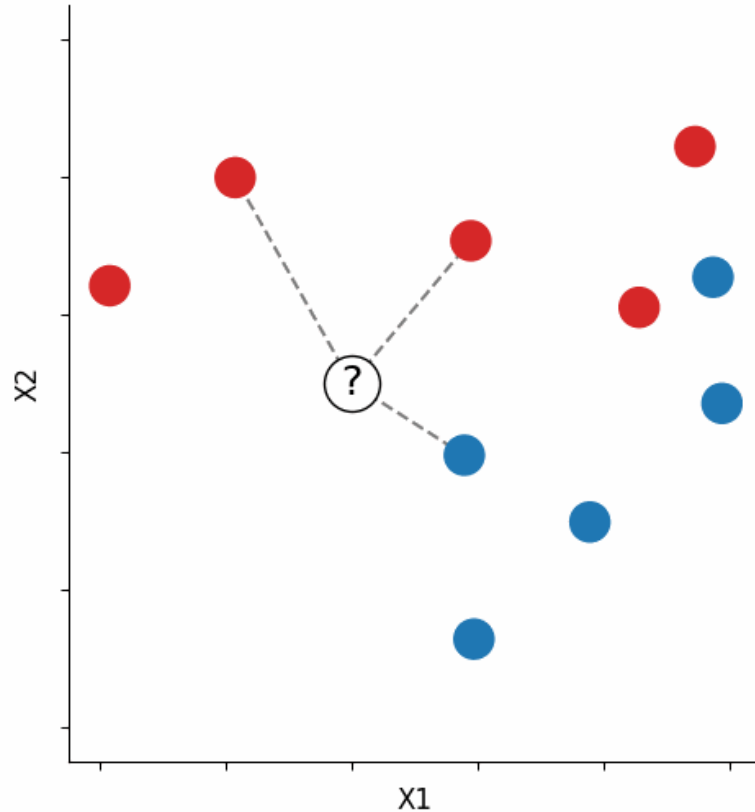
KNN Nasıl Çalışır?

- KNN algoritmasının çalışma adımları şöyledir:
- Yeni bir veri noktası alınır
- Eğitim verisindeki tüm örneklerle olan uzaklık hesaplanır
- En yakın **K adet komşu** seçilir
- Bu komşuların sınıfları incelenir
- Çoğunluk hangi sınıftaysa, yeni veri o sınıfa atanır
- Diyelim ki elimizde iki sınıf olsun:
 - Mavi sınıf
 - Kırmızı sınıf
- Yeni gelen bir noktanın çevresindeki 5 en yakın komşuya bakalım:
 - 3 tanesi mavi
 - 2 tanesi kırmızı
- Bu durumda:
 - $K = 5$
Sonuç = Mavi
- Yani model karar verirken:
 - “Bana en çok benzeyenler hangi sınıfta?” sorusunu kullanır.

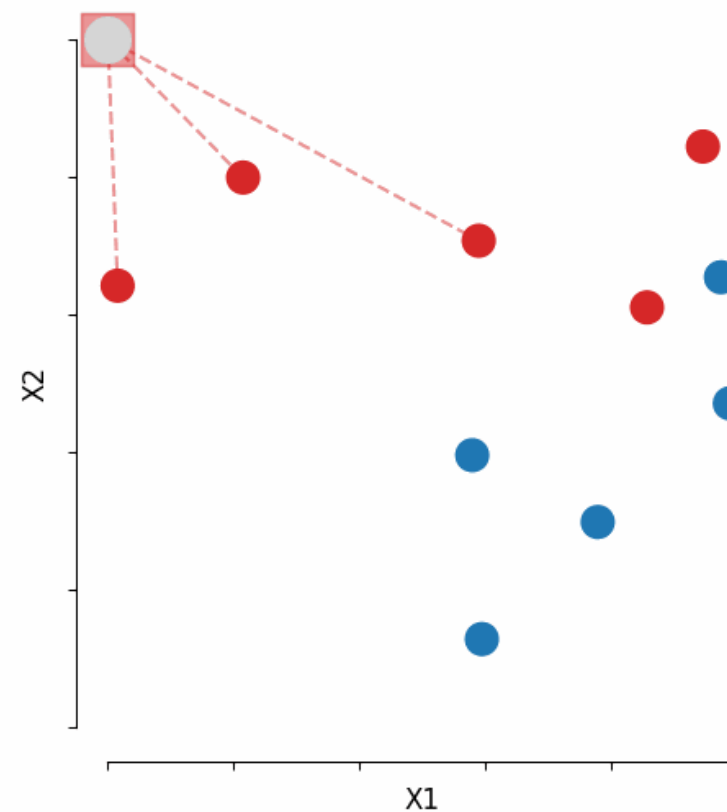
KNN Nasıl Çalışır?

kNN classifier | $k = 3$

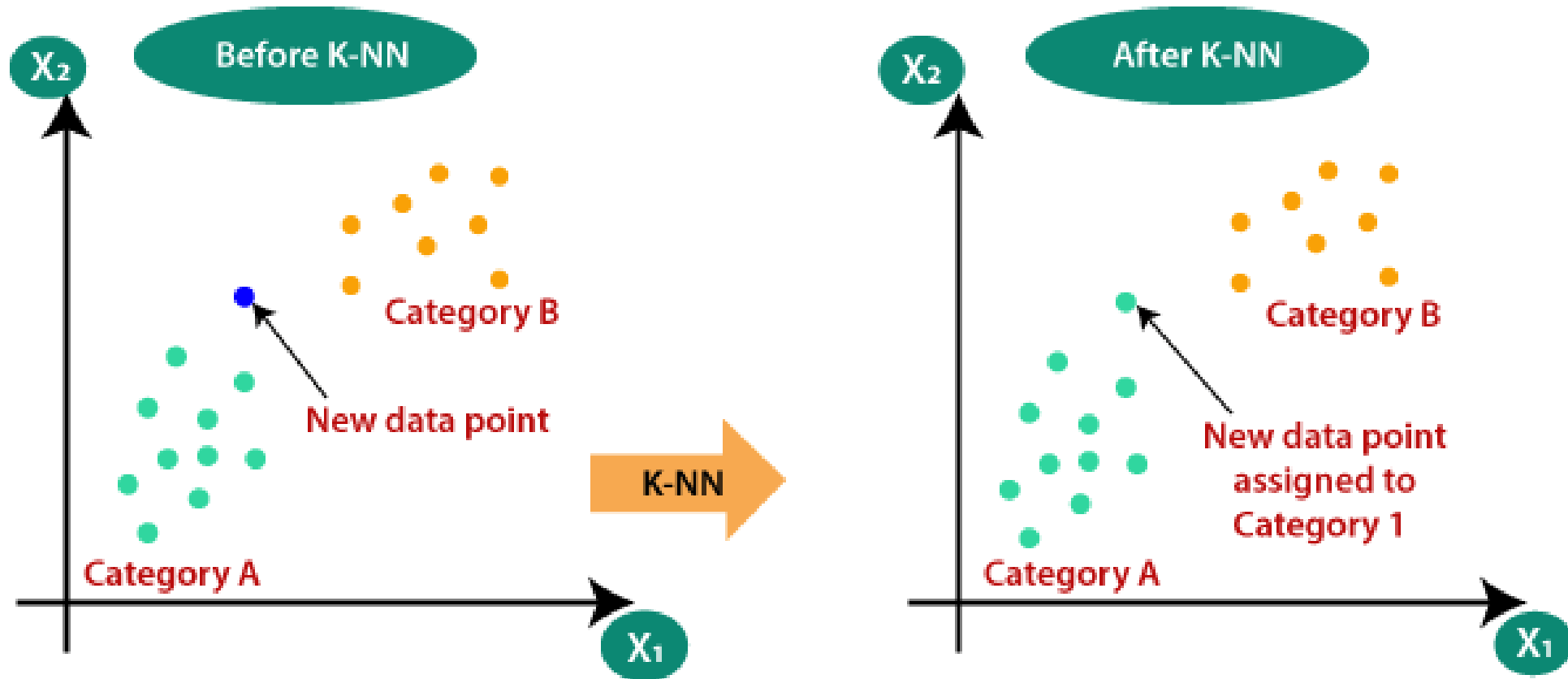
Classifying a single test point



Drawing the decision surface



KNN Nasıl Çalışır?



KNN Neden “Öğrenmiyor” Gibi Görünür?

- KNN, diğer bazı algoritmalar gibi karmaşık bir model üretmez.
- Bu yüzden KNN için şu ifade kullanılır:
 - **Lazy Learner (Tembel Öğrenici)**
- Yani:
 - eğitim aşamasında çok büyük bir model kurmaz
 - veriyi hafızasında tutar
 - tahmin aşamasında hesaplama yapar
- Bu yönüyle Decision Tree’den farklıdır.

KNN İçin En Kritik Kavram: Mesafe

- KNN'nin başarısı büyük ölçüde şu soruya bağlıdır:
 - “Yakınlık nasıl ölçülecek?”
- Bu yüzden KNN’de **distance (mesafe)** kavramı çok önemlidir.
- En yaygın mesafe türleri:
 - Euclidean Distance
 - Manhattan Distance
 - Minkowski Distance

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

Euclidean ve Manhattan mesafesinin genelleştirilmiş hâlidir.

Scikit-learn içinde KNN varsayılan olarak çoğu zaman Minkowski kullanır.

K Değeri Nedir, K Değeri Neden Önemlidir?

- KNN’de **K**, dikkate alınacak komşu sayısıdır.
- Örneğin:
 - $K = 1 \rightarrow$ yalnızca en yakın komşuya bak
 - $K = 3 \rightarrow$ en yakın 3 komşuya bak
 - $K = 5 \rightarrow$ en yakın 5 komşuya bak
- **K çok küçükse**
 - model gürültüye duyarlı olur
 - overfitting oluşabilir
- **K çok büyükse**
 - model fazla genelleme yapar
 - underfitting oluşabilir
- Bu yüzden uygun K değeri seçilmelidir.

KNN

- **KNN'nin Avantajları**

- mantığı çok basittir
- uygulaması kolaydır
- küçük veri setlerinde etkili olabilir
- eğitim aşaması hızlıdır

- **KNN'nin Dezavantajları**

- büyük veri setlerinde yavaş olabilir
- tüm veri noktalarına mesafe hesaplar
- ölçek farklarından çok etkilenir
- bu yüzden «StandarScaler», «MinMaxScaler» **özellik ölçekleme (scaling)** önemlidir

Decision Tree ile KNN Arasındaki Fark

Özellik	Decision Tree	KNN
Model yapısı	Kural tabanlı ağaç	Mesafe tabanlı
Eğitim	Model kurar	Veriyi saklar
Tahmin	Hızlı	Daha yavaş
Ölçekleme ihtiyacı	Genelde gerekmez	Çok önemlidir
Yorumlanabilirlik	Yüksek	Düşük

BİTTİ 😊